

NODE JS – EXPRESS - EJS



QiKbyte - Programação e Banco de dados Home Sobre

EJS view engine!

Cursos

Descrição	Carga Horária
Lógica de Programação	40
Javascript	40
Banco de dados	40
NodeJs	20

QiKbyte - Programação e Banco de Dados - © Copyright 2021

EJS: linguagem de modelagem para criação de páginas HTML utilizando JavaScript. Ajuda a separar as views dos scripts de controle.

**O PROJETO AO LADO
SERÁ CRIADO USANDO
NODE + EXPRESS + EJS
+ BOOTSTRAP**

Professor Flávio Mota

NODE JS – EXPRESS - EJS



Trabalhando com o EJS view engine

1º PASSO - Vamos criar a seguinte estrutura de pastas e arquivos para nosso projeto.

Estrutura
de
Arquivos

```
> node_modules
├── views
│   ├── pages
│   │   ├── index.ejs
│   │   └── sobre.ejs
│   └── partials
│       ├── footer.ejs
│       ├── head.ejs
│       └── header.ejs
├── app.js
├── package-lock.json
└── package.json
```

Após criar a estrutura de pastas e arquivos, vamos executar o comando **npm init -y** para gerar o arquivo package.json

Vamos instalar as dependências do nosso projeto, digite:

```
npm install express --save
npm install ejs --save
```



Trabalhando com o EJS view engine

2º PASSO p1 - Vamos configurar o arquivo app.js

```
//define uma instância para o express
var express = require('express');
var app = express();
//define o EJS como nossa view engine
app.set('view engine', 'ejs');
```

Configurando a instancia do express e setando o EJS como view engine do nosso projeto



Trabalhando com o EJS view engine

2º PASSO p2 - Vamos configurar o arquivo app.js

```
// Rota principal (renderiza index.ejs)
app.get('/', function(req, res) {
  // Criação do objeto cursos que está armazenando a descrição
  // e carga horária dos cursos
  var cursos = [{ descricao: 'Lógica de Programação ', ch: 40 },
    { descricao: 'Javascript', ch: 40 },
    { descricao: 'Banco de dados', ch: 40 },
    { descricao: 'NodeJs', ch: 20 }
  ];
  // Método que vai renderizar o arquivo index com os dados
  // do objeto cursos
  res.render('pages/index', {
    cursos: cursos
  });
});
```

Configurando a rota principal '/' do projeto que renderiza o arquivo index.ejs que vai receber o objeto (cursos) com alguns dados já preenchidos

NODE JS – EXPRESS - EJS



Trabalhando com o EJS view engine

2º PASSO p3 - Vamos configurar o arquivo app.js

```
//Rota sobre (renderiza sobre.ejs)
app.get('/sobre', function(req, res) {
  res.render('pages/sobre');
});
app.listen(3000);
console.log('Rodando na porta 3000');
```

Configuração da rota '/sobre' que renderiza o arquivo sobre.ejs, definimos que a aplicação utilizará a porta 3000 para rodar nossa aplicação. É importante lembrar que o `res.render()` irá procurar nossas páginas dentro do diretório `views`.



Trabalhando com o EJS view engine

Partials: head.ejs, header.ejs, e footer.ejs

As partials são pedaços de códigos que serão comuns a todos as páginas do nosso projeto, elas vão compor o nosso frontend. Como na maioria das aplicações que construímos, muito código é reutilizado. Vamos definir três *partials* que utilizaremos em todo nosso site: *head.ejs*, *header.ejs*, e *footer.ejs*.

A screenshot of a file explorer interface. It shows a folder named 'partials' which is expanded. Inside the folder, there are three files listed: 'footer.ejs', 'head.ejs', and 'header.ejs'. Each file name is preceded by a green icon representing a file.



Trabalhando com o EJS view engine

3º PASSO p1 – Criando a partial head.ejs e configurando o framework BOOTSTRAP

Partials: head.ejs

```
<!-- views/partials/head.ejs -->
<meta charset="UTF-8">
<title> -- QiKbyte -- </title>
<!-- Bootstrap CSS -->
<link rel="stylesheet" href="//maxcdn.bootstrapcdn.com/bootstrap/3.2.0/css/bootstrap.min.css">
<style>
  body {
    padding-top: 50px;
  }
</style>
```



Trabalhando com o EJS view engine

3º PASSO p2 – Criando a partial header.ejs

Partials: header.ejs

```
<!-- views/partials/header.ejs -->
<nav class="navbar navbar-default" role="navigation">
  <div class="container-fluid">

    <div class="navbar-header">
      <a class="navbar-brand" href="#">
        Qikbyte - Programação e Banco de dados
      </a>
    </div>

    <ul class="nav navbar-nav">
      <li><a href="/">Home</a></li>
      <li><a href="/sobre">Sobre</a></li>
    </ul>

  </div>
</nav>
```



Trabalhando com o EJS view engine

3º PASSO p3 – Criando a partial footer.ejs

Partials: footer.ejs

```
<!-- views/partials/footer.ejs -->  
  
<p class="text-center text-muted">Qikbyte - Programação e Banco de Dados - © Copyright 2021</p>
```



Trabalhando com o EJS view engine

4º PASSO p1 – Configurando e compondo o arquivo index.ejs

Agora que temos nossas *partials* definidas, tudo o que precisamos é carregar os arquivos dentro do nosso index.ejs. Para incluir uma *partial* basta digitar o seguinte código:

<%- include('../partials/nomeDoArquivo.ejs') %>

index.ejs

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <%- include('../partials/head') %>
</head>
<body class="container">
  <header>
    <%- include('../partials/header') %>
  </header>
  <main>
    <div class="jumbotron">
      <h1>EJS view engine!</h1>
    </div>
    <h2>Cursos</h2>
  </main>
</body>
</html>
```



Trabalhando com o EJS view engine

4º PASSO p2 – Configurando e compondo o arquivo index.ejs

Agora que temos nossas *partials* definidas, tudo o que precisamos é carregar os arquivos dentro do nosso index.ejs. Para incluir uma *partial* basta digitar o seguinte código:

<%- include('../partials/nomeDoArquivo.ejs') %>

index.ejs

```
        <% }); %>
        </tbody>
    </table>
</main>
<footer>
    <%- include('../partials/footer') %>
</footer>
</body>
</html>
```



Trabalhando com o EJS view engine

4º PASSO p2 – LISTANDO OS DADOS NA VIEW INDEX.EJS

```
<table class="table table-bordered">
  <th>
    Descrição
  </th>
  <th>
    Carga Horária
  </th>
  <tbody>
    <% cursos.forEach(function(cursos){ %>
      <tr>
        <td><%= cursos.descricao %></td>
        <td><%= cursos.ch %></td>
      </tr>
    <% }); %>
  </tbody>
</table>
```

Esse bloco de código mostra o poder do EJS, onde podemos trazer as instruções do Javascript para dentro de um arquivo HTML, tornando nossa páginas mais dinâmica, nesse exemplo estamos criando uma tabela em tempo de execução, inserindo os dados do objeto cursos que veio do nosso backend direto na table.

index.ejs

NODE JS – EXPRESS - EJS

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <%- include('../partials/head') %>
</head>
<body class="container">
  <header>
    <%- include('../partials/header') %>
  </header>
  <main>
    <div class="row">
      <div class="col-sm-12">
        <div class="well">
          <h4> A QIKbyte é especializada em técnicas e tecnologias para iniciantes na
            área de Desenvolvimento de Sistemas.
            Especialistas em LÓGICA DE PROGRAMAÇÃO, PROGRAMAÇÃO ESTRUTURADA E ORIENTADA A OBJETOS
            (Javascript, C#, Java) BANCO DE DADOS (CONCEITUAL, LÓGICO E FÍSICO) Linguagem SQL e Banco
            de dados orientado a documentos (MongoDB) 🙌
            CONFIRA NOSSOS MATERIAIS E CURSOS!!
          </h4>
        </div>
      </div>
    </div>
  </main>
  <footer>
    <%- include('../partials/footer') %>
  </footer>
</body>
</html>
```

<%= EJS %>

Embedded JavaScript templating.

4º PASSO p3 – VIEW SOBRE.EJS

No arquivo sobre.ejs temos a inclusão das partials da mesma forma que a pagina index.ejs conforme o código abaixo.

```
<%-
include('../partials/nomeDo
Arquivo.ejs') %>
```

sobre.ejs

Professor Flávio Mota

Resultado FINAL DO NOSSO PROJETO

QiKbyte - Programação e Banco de dados Home Sobre

EJS view engine!

Cursos

Descrição	Carga Horária
Lógica de Programação	40
Javascript	40
Banco de dados	40
NodeJs	20

QiKbyte - Programação e Banco de Dados - © Copyright 2021